

To create simple decorator: *nestjs decorator Roles*

```
import { Reflector } from '@nestjs/core';

export const Roles = Reflector.createDecorator<String[]>()

*****

import { CanActivate, ExecutionContext, Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { Observable } from 'rxjs';
import { Roles } from './roles.decorator';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(
    context: ExecutionContext,
  ): boolean | Promise<boolean> | Observable<boolean> {
    const roles = this.reflector.get(Roles, context.getHandler())
    if(!roles) {
      return true
    }

    const request = context.switchToHttp().getRequest()
    const t1 = request.headers['roles']
    console.log("the roles of the user is: ", t1)
    return true;
  }
}
```

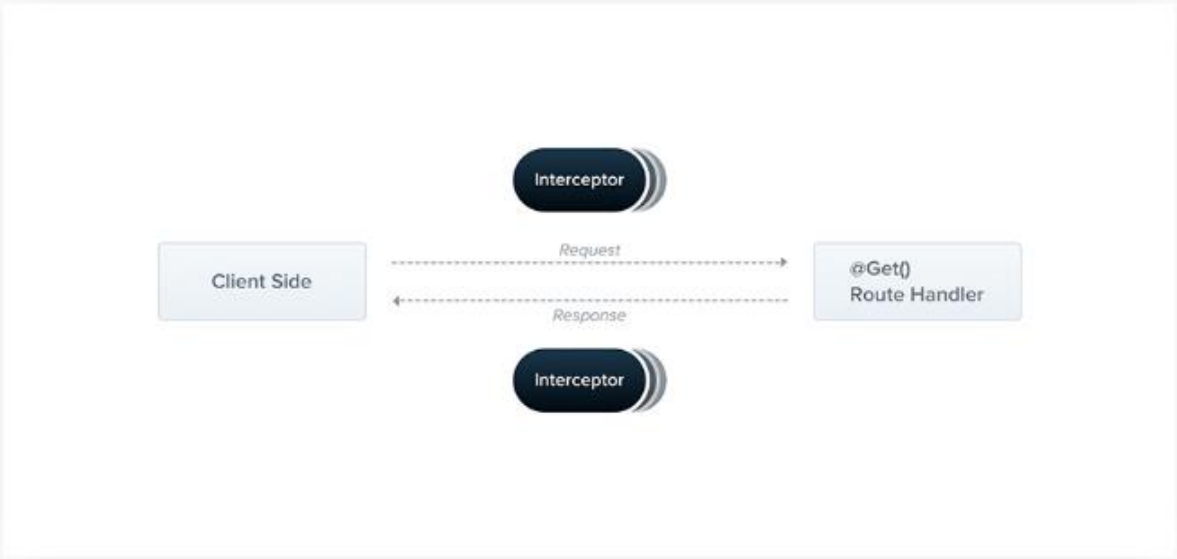
Note that behind the scenes, when a guard returns `false` , the framework throws a `ForbiddenException` . If you want to return a different error response, you should throw your own specific exception. For example:

```
throw new UnauthorizedException();
```

Any exception thrown by a guard will be handled by the **exceptions layer** (global exceptions filter and any exceptions filters that are applied to the current context).

Interceptors

An interceptor is a class annotated with the `@Injectable()` decorator and implements the `NestInterceptor` interface.



Interceptors have a set of useful capabilities which are inspired by the **Aspect Oriented Programming** (AOP) technique. They make it possible to:

- bind extra logic before / after method execution
- transform the result returned from a function
- transform the exception thrown from a function
- extend the basic function behavior
- completely override a function depending on specific conditions (e.g., for caching purposes)

Each `interceptor` implements the `intercept()` method, which takes two arguments. The first one is the `ExecutionContext` instance

The second argument is a `CallHandler`. The `CallHandler` interface implements the `handle()` method, which you can use to invoke the route handler method at some point in your interceptor. If you don't call the `handle()` method in your implementation of the `intercept()` method, the route handler method won't be executed at all.

Because the `handle()` method returns an `Observable`, we can use powerful **RxJS** operators to further manipulate the response. Using Aspect Oriented Programming terminology, the invocation of the route handler (i.e., calling `handle()`) is called a **Pointcut**, indicating that it's the point at which our additional logic is inserted.

To generate simple interceptor: *nest g interceptor Logging*

```
import { CallHandler, ExecutionContext, Injectable, NestInterceptor } from
 '@nestjs/common';
import { Observable, tap } from 'rxjs';

@Injectable()
export class LoggingInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    console.log('Before calling...')
    const now = Date.now()
    return next.handle().pipe(tap(() => console.log(`Total time is: ${now -
Date.now()}ms`)));
  }
}
```

HINT

The `NestInterceptor<T, R>` is a generic interface in which `T` indicates the type of an `Observable<T>` (supporting the response stream), and `R` is the type of the value wrapped by `Observable<R>`.

NOTICE

Interceptors, like controllers, providers, guards, and so on, can **inject dependencies** through their `constructor`.

Binding interceptors

In order to set up the interceptor, we use the `@UseInterceptors()` decorator imported from the `@nestjs/common` package. Like **pipes** and **guards**, interceptors can be controller-scoped, method-scoped, or global-scoped.

cats.controller.ts

JS

```
@UseInterceptors(LoggingInterceptor)
export class CatsController {}
```

HINT

The `@UseInterceptors()` decorator is imported from the `@nestjs/common` package.

Global interceptors are used across the whole application, for every controller and every route handler. In terms of dependency injection, global interceptors registered from outside of any module (with `useGlobalInterceptors()`, as in the example above) cannot inject dependencies since this is done outside the context of any module. In order to solve this issue, you can set up an interceptor **directly from any module** using the following construction:

app.module.ts

JS

```
import { Module } from '@nestjs/common';
import { APP_INTERCEPTOR } from '@nestjs/core';

@Module({
  providers: [
    {
      provide: APP_INTERCEPTOR,
      useClass: LoggingInterceptor,
    },
  ],
})
export class AppModule {}
```

Response mapping

We already know that `handle()` returns an `Observable`. The stream contains the value **returned** from the route handler, and thus we can easily mutate it using RxJS's `map()` operator.

WARNING

The response mapping feature doesn't work with the library-specific response strategy (using the `@Res()` object directly is forbidden).

Here we can use constructor to inject the cache handler and check the values.

cache.interceptor.ts

JS

```
import { Injectable, NestInterceptor, ExecutionContext, CallHandler } from '@nestjs/core';
import { Observable, of } from 'rxjs';

@Injectable()
export class CacheInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const isCached = true;
    if (isCached) {
      return of([]);
    }
    return next.handle();
  }
}
```

timeout.interceptor.ts

JS

```
import { Injectable, NestInterceptor, ExecutionContext, CallHandler, RequestTimeoutException } from '@nestjs/common';
import { Observable, throwError, TimeoutError } from 'rxjs';
import { catchError, timeout } from 'rxjs/operators';

@Injectable()
export class TimeoutInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    return next.handle().pipe(
      timeout(5000),
      catchError(err => {
        if (err instanceof TimeoutError) {
          return throwError(() => new RequestTimeoutException());
        }
        return throwError(() => err);
      })
    );
  }
};
```

```
import { createParamDecorator, ExecutionContext } from '@nestjs/common';
```

```
export const User = createParamDecorator(
  (data: unknown, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    console.log(`The request is: ${request.path}`);
    return { id: 1, username: 'JLoka-01' };
  },
);
```

```
@Get('/user')
getLokaUser(@User() user) {
  return user
}
```

Working with pipes

Nest treats custom param decorators in the same fashion as the built-in ones (`@Body()` , `@Param()` and `@Query()`). This means that pipes are executed for the custom annotated parameters as well (in our examples, the `user` argument). Moreover, you can apply the pipe directly to the custom decorator:

```
JS
@Get()
async findOne(
  @User(new ValidationPipe({ validateCustomDecorators: true }))
  user: UserEntity,
) {
  console.log(user);
}
```

HINT

Note that `validateCustomDecorators` option must be set to true. `ValidationPipe` does not validate arguments annotated with the custom decorators by default.

Decorator composition

Nest provides a helper method to compose multiple decorators. For example, suppose you want to combine all decorators related to authentication into a single decorator. This could be done with the following construction:

auth.decorator.ts

JS

```
import { applyDecorators } from '@nestjs/common';

export function Auth(...roles: Role[]) {
  return applyDecorators(
    SetMetadata('roles', roles),
    UseGuards(AuthGuard, RolesGuard),
    ApiBearerAuth(),
    ApiUnauthorizedResponse({ description: 'Unauthorized' }),
  );
}
```

You can then use this custom `@Auth()` decorator as follows:

```
@Get('users')
@Auth('admin')
findAllUsers() {}
```

This has the effect of applying all four decorators with a single declaration.

WARNING

The `@ApiHideProperty()` decorator from the `@nestjs/swagger` package is not composable and won't work properly with the `applyDecorators` function.

To add jwt auth for our apis; first we install jwt: `npm i --save @nestjs/jwt`

```
import { Injectable } from '@nestjs/common';

// for real scenarios we must declare it using classes with class-validator
export type User = any;

@Injectable()
export class UsersService {

  private readonly users = [
    {
      userId: 1,
      username: 'Jafar',
      password: 'Loka',
    },
    {
      userId: 2,
      username: 'Maria',
      password: 'Loka',
    },
  ];

  async findOne(username: string): Promise<User | undefined> {
    return this.users.find(user => user.username === username);
  }
}
```

For auth module:

```
import { Module } from '@nestjs/common';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { UsersModule } from 'src/users/users.module';
import { JwtModule } from '@nestjs/jwt';
import { jwtConstants } from './constants';

@Module({
  controllers: [AuthController],
  providers: [AuthService],
  imports: [UsersModule, JwtModule.register({
    global: true,
```

```

        secret: jwtConstants.secret,
        signOptions: { expiresIn: '3600s' }
    ]],
    exports: [AuthService],
  })
  export class AuthModule {}

```

For auth controller:

```

import { Body, Controller, Get, HttpStatusCode, HttpStatus, Post, Request, UseGuards }
from '@nestjs/common';
import { AuthService } from '../auth.service';
import { SignInDto } from '../dto/signin.dto';
import { AuthGuard } from '../auth.guard';

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @HttpCode(HttpStatus.OK)
  @Post('login')
  signIn(@Body() signInDto: SignInDto) {
    return this.authService.signIn(signInDto.username, signInDto.password);
  }

  @UseGuards(AuthGuard)
  @Get('profile')
  async profile(@Request() req) {
    return req.user
  }
}

```

For auth service:

```
import { Get, Injectable, Request, UnauthorizedException, UseGuards } from
 '@nestjs/common';
import { UsersService } from 'src/users/users.service';
import { JwtService } from '@nestjs/jwt';
import { AuthGuard } from './auth.guard';

@Injectable()
export class AuthService {
  constructor(private readonly usersService: UsersService, private readonly
jwtService: JwtService) {}

  async signIn(username: string, pass: string): Promise<{ access_token: string
}> {
    const user = await this.usersService.findOne(username)
    if(user.password !== pass) {
      throw new UnauthorizedException();
    }

    const payload = { sub: user.userId, username: user.username }
    return {
      access_token: await this.jwtService.signAsync(payload),
    }
  }
}
```

For constants:

```
export const jwtConstants = {
  secret: 'JLoka@123'
}
```

For auth guard:

```
import { CanActivate, ExecutionContext, Injectable, UnauthorizedException } from
 '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { Request } from 'express';
import { jwtConstants } from './constants';
@Injectable()
export class AuthGuard implements CanActivate {
```

```

constructor(private readonly jwtService: JwtService){}

async canActivate(
  context: ExecutionContext,
): Promise<boolean> {

  const request = context.switchToHttp().getRequest()
  const token = this.extractTokenFromHeaders(request);
  if(!token) {
    throw new UnauthorizedException()
  }

  try {
    const payload = await this.jwtService.verifyAsync(token, {
      secret: jwtConstants.secret,
    })
    request["user"] = payload
  } catch {
    throw new UnauthorizedException()
  }

  return true;
}

extractTokenFromHeaders(request: Request): string|undefined {
  const [type, token] = request.headers["authorization"]?.split(' ') ?? []
  return type == 'Bearer' ? token : undefined
}
}

```